

# ● SESIÓN 4 Validación de datos (Bean Validation)

## + errores coherentes

"No confiamos en el cliente."

## 1. Estructura del proyecto (Sesión 4)

```
com.cladsb1.books
├── controller
│   └── BookController
├── entity
│   └── Book
├── repository
│   └── BookRepository
├── dto
│   └── request
│       └── BookRequestDTO
├── error
│   ├── ApiError
│   └── GlobalExceptionHandler
└── BooksApplication
```

Esta estructura organiza claramente las responsabilidades: los DTOs definen qué aceptamos desde fuera, mientras que el paquete error centraliza el manejo de excepciones.

## 2. Dependencia necesaria

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

❏ **Importante:** Sin esta dependencia, las anotaciones de validación como @NotBlank y @Size no funcionarán.

### 3. DTO de entrada con validaciones

📁 dto/request/BookRequestDTO.java

```
package com.cladsb1.books.dto.request;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Size;

/**
 * DTO de entrada:
 * - Define QUÉ aceptamos desde fuera
 * - Aquí viven las reglas de validación
 */
public record BookRequestDTO(
    @NotBlank(message = "Title is required")
    @Size(min = 2, max = 120, message = "Title must be between 2 and 120 characters")
    String title,

    @NotBlank(message = "Author is required")
    @Size(min = 2, max = 80, message = "Author must be between 2 and 80 characters")
    String author,

    @NotBlank(message = "Category is required")
    @Size(min = 2, max = 40, message = "Category must be between 2 and 40 characters")
    String category
) {}
```

#### @NotBlank

Valida que el campo no esté vacío  
ni contenga solo espacios

#### @Size

Define límites mínimos y  
máximos de caracteres

#### message

Personaliza el mensaje de error  
que verá el cliente

## 4. ApiError (formato estándar)

 error/ApiError.java

```
package com.cladsb1.books.error;

import java.time.Instant;
import java.util.Map;

/**
 * Respuesta de error estándar para la API.
 * - fieldErrors se usa especialmente en validaciones
 */
public record ApiError(
    Instant timestamp,
    int status,
    String error,
    String path,
    Map<String, String> fieldErrors
) {}
```

Este record define un formato consistente para todos los errores de la API. El campo `fieldErrors` es especialmente útil para validaciones, ya que permite devolver múltiples errores de campos simultáneamente.

## 5. GlobalExceptionHandler (DOS VERSIONES)

✧ VERSIÓN MODERNA



### A) Versión MODERNA (CON streams/lambda)

📁 **error/GlobalExceptionHandler.java (CON streams)**

```
package com.cladsb1.books.error;

import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.*;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;

import java.time.Instant;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.stream.Collectors;

/**
 * Manejo global de errores:
 * - Centraliza respuestas de error
 * - Evita ensuciar controllers
 */
@RestControllerAdvice
public class GlobalExceptionHandler {

    /**
     * Cuando falla @Valid en un @RequestBody, Spring lanza MethodArgumentNotValidException.
     * Aquí lo transformamos en un 400 con un JSON claro para el cliente.
     */
    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError> handleValidation(MethodArgumentNotValidException ex,
        HttpServletRequest request) {
        // Versión con streams:
        // - tomamos los FieldError
        // - los convertimos a Map<campo, mensaje>
        // - si hay duplicados, nos quedamos con el primero
        Map<String, String> errors = ex.getBindingResult().getFieldErrors()
            .stream()
            .collect(Collectors.toMap(
                FieldError::getField,
                FieldError::getDefaultMessage,
                (a, b) -> a,
                LinkedHashMap::new
            ));

        ApiError body = new ApiError(
            Instant.now(),
            HttpStatus.BAD_REQUEST.value(),
            "Validation failed",
            request.getRequestURI(),
            errors
        );

        return ResponseEntity.badRequest().body(body);
    }
}
```



## B) Versión CLÁSICA (SIN streams/lambda)

📁 error/GlobalExceptionHandler.java (SIN streams)

```
package com.cladsb1.books.error;

import jakarta.servlet.http.HttpServletRequest;
import org.springframework.http.*;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;

import java.time.Instant;
import java.util.LinkedHashMap;
import java.util.Map;

/**
 * Manejo global de errores (sin streams).
 */
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ApiError> handleValidation(MethodArgumentNotValidException ex,
                                                    HttpServletRequest request) {
        Map<String, String> errors = new LinkedHashMap<>();

        // Versión clásica con for:
        for (FieldError fe : ex.getBindingResult().getFieldErrors()) {
            // Si quieres evitar sobrescribir en caso de duplicados:
            if (!errors.containsKey(fe.getField())) {
                errors.put(fe.getField(), fe.getDefaultMessage());
            }
        }

        ApiError body = new ApiError(
            Instant.now(),
            HttpStatus.BAD_REQUEST.value(),
            "Validation failed",
            request.getRequestURI(),
            errors
        );

        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(body);
    }
}
```



👉 **Elige una versión.** No mantengas las dos a la vez porque duplicarías el handler.



# 6. Controller (DOS VERSIONES): CON lambdas/Optional.map y SIN lambdas

Aquí sí aplica lo que hicimos en sesión 3: Optional + 404, pero ahora con @Valid y DTO.

## A) Controller MODERNO (CON lambdas)

 controller/BookController.java (CON lambdas)

```
package com.cladsb1.books.controller;

import com.cladsb1.books.dto.request.BookRequestDTO;
import com.cladsb1.books.entity.Book;
import com.cladsb1.books.repository.BookRepository;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.net.URI;
import java.util.List;

@RestController
@RequestMapping("/api/books")
public class BookController {

    private final BookRepository bookRepository;

    public BookController(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    @GetMapping
    public ResponseEntity<List<Book>> getAll() {
        return ResponseEntity.ok(bookRepository.findAll());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Book> getById(@PathVariable Long id) {
        return bookRepository.findById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    /**
     * CREATE:
     * - Recibimos DTO validado (@Valid)
     * - Construimos la Entity
     * - Guardamos
     * - Respondemos 201 + Location
     */
    @PostMapping
    public ResponseEntity<Book> create(@Valid @RequestBody BookRequestDTO dto) {
        Book book = new Book(dto.title(), dto.author(), dto.category());
        Book saved = bookRepository.save(book);
        URI location = URI.create("/api/books/" + saved.getId());
        return ResponseEntity.created(location).body(saved);
    }

    /**
     * UPDATE:
     * - 404 si no existe
     * - 400 si el DTO es inválido
     */
    @PutMapping("/{id}")
    public ResponseEntity<Book> update(@PathVariable Long id,
        @Valid @RequestBody BookRequestDTO dto) {
        return bookRepository.findById(id)
            .map(existing -> {
                existing.setTitle(dto.title());
                existing.setAuthor(dto.author());
                existing.setCategory(dto.category());
                Book saved = bookRepository.save(existing);
                return ResponseEntity.ok(saved);
            })
            .orElse(ResponseEntity.notFound().build());
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        if (!bookRepository.existsById(id)) {
            return ResponseEntity.notFound().build();
        }
        bookRepository.deleteById(id);
        return ResponseEntity.noContent().build();
    }
}
```



## B) Controller CLÁSICO (SIN lambdas)

 **controller/BookController.java (SIN lambdas)**

```
package com.cladsb1.books.controller;

import com.cladsb1.books.dto.request.BookRequestDTO;
import com.cladsb1.books.entity.Book;
import com.cladsb1.books.repository.BookRepository;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.net.URI;
import java.util.List;
import java.util.Optional;

@RestController
@RequestMapping("/api/books")
public class BookController {

    private final BookRepository bookRepository;

    public BookController(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    @GetMapping
    public ResponseEntity<List<Book>> getAll() {
        List<Book> books = bookRepository.findAll();
        return ResponseEntity.ok(books);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Book> getById(@PathVariable Long id) {
        Optional<Book> opt = bookRepository.findById(id);
        if (opt.isEmpty()) {
            return ResponseEntity.notFound().build();
        }
        return ResponseEntity.ok(opt.get());
    }

    @PostMapping
    public ResponseEntity<Book> create(@Valid @RequestBody BookRequestDTO dto) {
        Book book = new Book(dto.title(), dto.author(), dto.category());
        Book saved = bookRepository.save(book);
        URI location = URI.create("/api/books/" + saved.getId());
        return ResponseEntity.created(location).body(saved);
    }

    @PutMapping("/{id}")
    public ResponseEntity<Book> update(@PathVariable Long id,
        @Valid @RequestBody BookRequestDTO dto) {
        Optional<Book> opt = bookRepository.findById(id);
        if (opt.isEmpty()) {
            return ResponseEntity.notFound().build();
        }

        Book existing = opt.get();
        existing.setTitle(dto.title());
        existing.setAuthor(dto.author());
        existing.setCategory(dto.category());
        Book saved = bookRepository.save(existing);
        return ResponseEntity.ok(saved);
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> delete(@PathVariable Long id) {
        boolean exists = bookRepository.existsById(id);
        if (!exists) return ResponseEntity.notFound().build();

        bookRepository.deleteById(id);
        return ResponseEntity.noContent().build();
    }
}
```



## 7. Tabla: caso → status (para Swagger)

| Endpoint            | Caso         | Qué lo provoca  | Status |
|---------------------|--------------|-----------------|--------|
| POST /api/books     | válido       | pasa @Valid     | 201    |
| POST /api/books     | inválido     | @NotBlank/@Size | 400    |
| PUT /api/books/{id} | id no existe | repo vacío      | 404    |
| PUT /api/books/{id} | dto inválido | falla @Valid    | 400    |
| GET /api/books/{id} | id no existe | repo vacío      | 404    |

## 8. Prueba rápida en Swagger

POST con title=""  
→ 400 con JSON ApiError

PUT a id inexistente  
→ 404

POST correcto  
→ 201 con Location

## 9. Errores típicos

Olvidar spring-boot-starter-validation  
→ no valida

Validar en entity en vez de DTO  
→ acopla API a BD

Olvidar @Valid  
→ no valida

No tener handler  
→ 400 "feo" (sin formato)